

Diagrama UML: sistema de biblioteca

Luisa Fernanda García Álvarez

Natalia Andrea Amaya García

Leandro Marín Parra

Docente

Boris Alberto Salleg

Orientación Orientada a Objetos Avanzada

Institución Universitaria Digital de Antioquia

30/08/2026

Introducción

Ese trabajo nace de la necesidad de entender, de manera práctica, cómo se organiza la información antes de convertirla en un programa. Para lograrlo, elegimos un sistema de bibliotecas con sus libros, sus autores y los préstamos que se hacen a los usuarios.

La idea principal fue representar, a través de un diagrama de clases cómo se relacionan estos elementos entre sí: Qué información maneja cada uno, que hace cada parte del sistema y cómo se conectan unos a otros. Este ejercicio nos permitió aplicar de forma sencilla algunos principios importantes de la programación orientada a objetos, como organizar bien la información, evitar que las partes del sistema dependen demasiado una de otras y dejar todo diseñado de manera clara para que si en un futuro se requiere ampliar o modificar sea fácil de hacer.

Más allá de lo técnico, este trabajo también fue una oportunidad para pensar como equipo la mejor forma de resolver un problema real, cómo diseñar paso a paso algo que en la vida cotidiana damos por hecho, como el simple acto de pedir prestado un libro.

Objetivo General

Diseñar un diagrama UML que permita mostrar de manera clara cómo está organizado y cómo funciona un sistema de biblioteca, teniendo en cuenta las características principales de la programación orientada a objetos. (abstracción, encapsulamiento, herencia y polimorfismo).

Objetivos específicos

- Identificar las clases, los datos y las acciones necesarias para representar el sistema de biblioteca.
- Definir cómo se relacionan las diferentes clases de acuerdo con la función que cumple cada uno dentro del sistema.
- Explicar y registrar el proceso de creación del diagrama de manera clara, trabajando de forma colaborativa.

Descripción de las clases identificadas

El sistema modela una biblioteca a partir de cinco clases principales: Usuario, Préstamo, Libro, Libro Digital y Autor. Cada una encapsula sus atributos como privados y expone su comportamiento únicamente a través de métodos públicos, cumpliendo el principio de encapsulamiento. Usuario representa a la persona que solicita libros en préstamo, Libro representa el material físico disponible en el catálogo, con su especialización Libro Digital, para recursos electrónicos; Autor almacena la información de quien escribió cada obra y préstamo es la clase que articula la operación central del sistema, vinculando a un usuario con un libro durante un periodo determinado.

La decisión de separar préstamo como clase independiente, en lugar de manejarlo como simple atributo de Libro o Usuario, responde al principio de responsabilidad única (S de SOLID): cada clase se encarga exclusivamente de su propio dominio de información y comportamiento, sin mezclar la lógica de disponibilidad de un libro con la lógica temporal de un préstamo.

Justificación de las relaciones

Asociación (usuario-préstamo): un usuario puede realizar varios préstamos a lo largo de tiempo ($1^a < 0 \dots *$), pero ambas clases tienen existencia independiente entre sí, un usuario sigue existiendo, aunque no tenga préstamos activos.

Composición (préstamo-libro): se modela como composición por qué un préstamo no tiene sentido sin el libro que referencia-la relación es de dependencia fuerte de todo (préstamo) hacia su parte Libro, representada con el rombo relleno. Esto refleja que el ciclo de vida de la instancia de préstamo está intrínsecamente ligado a la existencia de un libro concreto en ese momento.

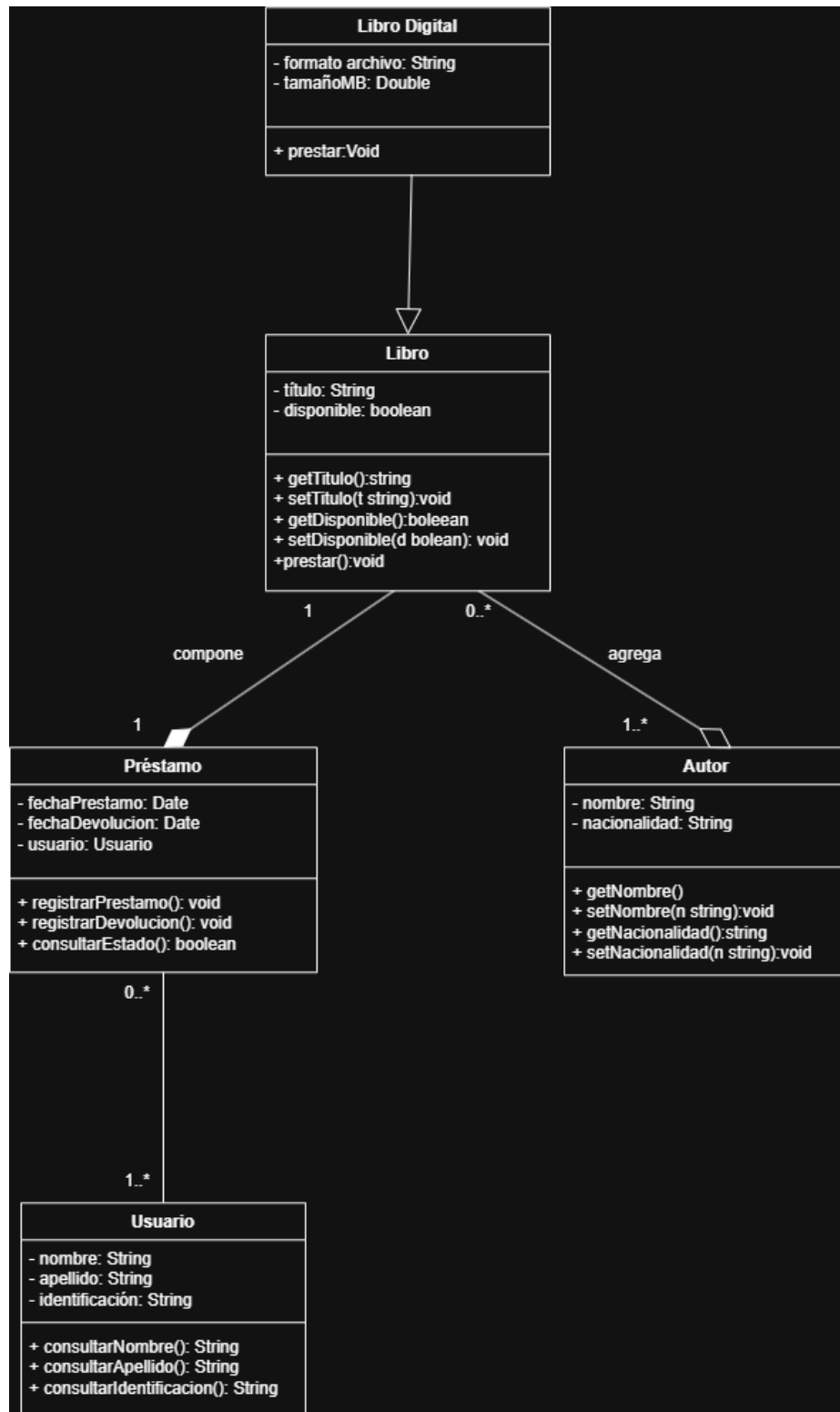
Agregación (Autor-Libro): un autor puede agregar varios libros que ha escrito (1 a 0..*), pero a diferencia de la composición, aquí la relación es débil: si se elimina un autor del sistema, los libros pueden seguir existiendo en el catálogo (por ejemplo, reasignados o conservados con fines históricos). Esto justifica el uso del rombo vacío en lugar del relleno.

Herencia (Libro-Libro Digital): hereda del Libro los atributos título y disponible, así como los métodos base, pero sobrescribe el método prestar () para reflejar un comportamiento distinto: un libro físico bloquea su disponibilidad al prestarse, mientras que un libro digital genera una copia de acceso temporal sin agotar existencias. Esta sobreescritura es un ejemplo directo de polimorfismo: el sistema puede invocar prestar () sobre cualquier objeto de tipo libro sin conocer si en tiempo de ejecución se trata de un Libro Digital, y cada uno responde según su propia implementación.

Cohesión, acoplamiento y principio SOLID

El diseño mantiene alta cohesión porque cada clase agrupa únicamente los atributos y métodos relacionados con su propia responsabilidad. El acoplamiento es bajo gracias a polimorfismo entre Libro Digital: La clase préstamo puede componerse de cualquier subtipo de Libro sin necesitar conocer los detalles internos de cada uno, lo que también evidencia el principio abierto/cerrado el sistema queda abierto a nuevas extensiones (por ejemplo, una futura clase Libro Audio) sin necesidad de modificar el código ya existente de Libro o Préstamo.

Diagrama de clases UML para el sistema biblioteca



ENLACES

Diagrama: https://drive.google.com/file/d/1iTxn8P8MNz0EpUKX6XV_C_D5Ue-Gvdsr/view?usp=drive_link

Repositorio GitHub: <https://github.com/eslef1998/Diagrama-clases-biblioteca-grupo11>

Video: <https://drive.google.com/file/d/1RNs-FG-1aFJ2R4dwy1Xq6K0FI36Z6vaL/view?usp=sharing>

CONCLUSIONES

- En conclusión, con el desarrollo de este trabajo pude comprender de una manera más práctica cómo se estructura y organiza la información antes de llevarla a un programa. Al realizar el diagrama UML del sistema de biblioteca, logre identificar la función de cada clase y entender cómo sus relaciones permiten representar situaciones reales, como el registro de usuarios, autores, libros y préstamos.

Por otra parte, para mí fue sorprendente la manera en la que es posible, a partir de un diagrama de clases, comprender el mundo y sobre todo, un proceso. Pienso que a cualquier fenómeno de la vida se le pueden hacer diagramas de clases diversos y su diversidad dependerá de la solución que se le quiera dar. En resumen, posibilita integrar una mirada sistémica del mundo que nos ayude a ser profesionales con una fuerte inclinación hacia la resolución de problemas, y la creación de soluciones tecnológicas.

Finalmente, todo este ejercicio demostró el valor práctico de planificar la arquitectura de software y aplicar los principios de diseño (como SOLID) antes de iniciar a escribir las primeras líneas de código. notar como actos tan simples como el de separar la lógica de un proyecto evita enredos técnicos, me hizo entender también que el modelado previo no es un simple trámite, si no la clave para construir un sistema ordenado y escalable en el futuro.